

Next-Stroke Prediction in Table Tennis with Test-Time Player Profiling for Training-Unseen Players

Yi-Hsin Wang¹[0009–0009–3862–172X](✉) and Ya-You Shih²[0009–0008–6198–6578]

¹ Department of Computer Science and Information Engineering, National Taiwan Normal University, Taipei, Taiwan

41247055s@gapps.ntnu.edu.tw

² Interdisciplinary Program of Electrical Engineering and Computer Science, National Central University, Taoyuan, Taiwan

s112504506@g.ncu.edu.tw

Abstract. Predicting the next stroke in table tennis is challenging when only a few strokes are observed and players have not appeared in the training data. To supplement the limited rally context, we describe each player by cross-rally statistics of their observed strokes and feed the statistics of the next hitter and the opponent, with rally features, to gradient-boosted decision trees. Training-unseen players, however, lack such profiles. We address this gap through Test-Time Player Profiling (TTPP), which pools each player’s observed strokes across the visible prefixes of an offline prediction batch: each prefix is short, but across rallies the observations accumulate into behavioral information. TTPP touches no prediction target and, under our main training recipe, requires no retraining. Experiments use the AI CUP 2026 table tennis dataset with match-grouped cross-validation. Adding player statistics raises the overall score from 0.285 to 0.322; on the full feature set (0.345), TTPP adds a further 0.017 (to 0.362). Both improvements are positive in every fold across three splits, with TTPP gains concentrated in rallies involving players unseen in the training folds. Evaluating TTPP raises a further question: which strokes may go into the profiles of validation-set players? Building them from training data alone erases the TTPP gain; adding the strokes visible at inference gives the estimate we report; also adding the strokes still to be predicted inflates scores. On this dataset, pooling a player’s observed strokes across rallies profiles players absent from training, provided the method is evaluated with the observations available at inference.

Keywords: Table tennis tactical analysis · Next-stroke prediction · Training-unseen players · Gradient-boosted decision trees · Test-time player profiling · Validation design

1 Introduction

Next-stroke prediction—predicting the next stroke from the strokes already played in a rally—is a basic problem in the tactical analysis of racket sports

[3,15,16]. Given the first K strokes of a rally (the prefix), the task is to predict the type and placement of stroke $K + 1$ and whether the server wins the rally. A prediction can draw on two kinds of information: how the current rally has developed so far, and how the players involved tend to hit across rallies.

We study this task under two conditions. First, only a few strokes of the current rally are visible, in the setting we study typically one or two, so the current rally offers few cues about how the exchange will continue. Second, many of the players at deployment do not appear in the training data, so their past behavior cannot be read from the training set. Each condition closes off one source of information. The question we address is how to obtain useful player behavior information when the current rally is short and the player has no training record. Sequence models for this task represent players by embeddings learned during training and are evaluated with every player seen [16]; neither carries over to players who first appear at deployment (Section 2).

Our approach fills the gap in three steps. First, because a short rally says little on its own, we describe each player by statistics counted over their other rallies, such as stroke-type and placement distributions (we call a player’s statistics vector their profile), for both the next hitter and the opponent, and use them as features alongside the current-rally features in a gradient-boosted decision tree model [6]. Second, a player absent from training is not a zero-information cold start [11]: when a whole set of rallies is predicted at once (offline batch prediction), the same player typically appears in many of them, and the visible strokes of those rallies, short as each is, accumulate into a usable profile. Test-Time Player Profiling (TTPP) builds such profiles at inference from the unlabeled visible strokes, a transductive use of the test inputs [13], without touching the stroke to be predicted or the rally outcome, and without retraining. Third, because TTPP draws on the batch being predicted, validation has to reproduce the same information condition [5]: the profiles of the players in validation rallies may use the strokes that would be observable at inference, but not the strokes and outcomes that are to be predicted. Section 5.4 shows that this choice decides whether the TTPP gain can be observed at all.

The AI CUP 2026 table tennis dataset [1] contains both conditions and serves as the test bed: 53.3% of test rallies show at most two strokes, and 43.7% of test players, involved in 54.4% of test rallies, never appear in training but leave visible strokes in the test data (Section 3). We make three contributions. (1) **Player statistics for short prefixes**: we show that cross-rally statistics for both the next hitter and the opponent give the largest single-component gain (adding them raises Overall from 0.285 to 0.322; removing them from the final model lowers it by 0.027; both positive in every fold of three splits; Section 5.2). (2) **TTPP for training-unseen players**: an inference-time procedure that turns the unlabeled strokes visible in the deployment batch into player statistics; its gain concentrates in rallies with unseen players, and no retraining is needed (Section 5.3 also reports a stricter training variant under which retraining does help). (3) **Deployment-aligned validation**: validation rows also need player profiles, and we show that the choice of which strokes may go into them changes

the conclusion. Building them from training data alone, as standard out-of-fold validation does [7], erases the TTPP gain; adding the validation batch’s visible prefixes (strokes 1 to K) gives the estimate we report; also adding stroke $K + 1$ and the rally outcome, the very targets of prediction, inflates scores (Section 5.4). All main results are cross-validation estimates replicated on three match-grouped splits (Section 5.1).

2 Related Work

Stroke-level prediction in racket sports. Table tennis research has characterized tactical structure with statistical models [17,10,8], aiming at post-match analysis rather than stroke-level prediction. Hsu et al. [3] applied a ShuttleNet-style Transformer to table tennis next-stroke prediction, but with data centered on a single player and within-match splits. In badminton, ShuttleNet [16], MuLMINet [12], the ShuttleSet benchmark [15], and RallyTempo [4] established the task paradigm. Their player representations are learned during training, and their evaluation protocols (e.g., using the first 80% of rallies of each match for training [16]) ensure that all players are seen, so they cannot address the player-disjoint setting. We place the player representation at the feature level so that test-time observations can be incorporated.

Tabular models and test-time information. LightGBM [6] and CatBoost [9] are the mainstream GBDT implementations for tabular data [2]. We recast the problem as tabular because short prefixes leave little within-rally context, while player statistics are fixed-dimensional aggregates. TTPP is a direct application of transductive inference [13]; unlike test-time adaptation, which adjusts the model state [14], it only rebuilds input-side statistics and leaves the trained model untouched.

Cross-sample statistical features and information legitimacy. Experience with target encoding [7] shows that features aggregated from other samples inflate scores if they include the validation sample’s own label, which is why such features are built out-of-fold or by cross-fitting. Kaufman et al. [5] formalized this through “information legitimacy”: a feature is legitimate if the information it uses is available at deployment. We follow this view and show empirically how the choice of information policy changes the assessment of cross-sample features once statistics include test visible strokes (Section 5.4).

3 Task, Data, and Structural Properties

3.1 Task Definition and Evaluation Format

Given the first K strokes of a rally ($1 \leq K \leq L - 1$, where L is the rally length), the system predicts the type of stroke $K + 1$ (S1; of the 19 stroke-type classes of the annotation scheme, 15–18 are serve types that appear only at stroke 1, so the valid targets are the 15 classes 0–14), the placement of stroke $K + 1$ (S2; 10 classes, 0 = no legal placement), and whether the server wins the rally (S3). S1

Table 1. Dataset statistics

Item	Training	Test
Stroke records	84,707	5,668
Rallies	14,995	1,845
Matches	216	79 (disjoint from training)
Players	166	71 (40 seen in training, 31 unseen)
Rally length L / visible prefix K	mean 5.65 (median 5, max 52)	mean 3.07 (median 2, max 24)
Share of rallies with $K \leq 2$	—	53.3% ($K = 1$: 27.5%, $K = 2$: 25.7%)
Training-unseen players	—	31/71 (43.7%); 54.4% of rallies involve at least one

and S2 are scored by Macro-F1 and S3 by AUC; the overall score, as defined with the dataset, is $0.4 \cdot \text{F1}(\text{S1}) + 0.4 \cdot \text{F1}(\text{S2}) + 0.2 \cdot \text{AUC}(\text{S3})$. Macro-F1 is computed over a fixed class set (absent classes count as 0); the weighted score is called Overall, the dataset’s reference metric. Inference is batch-wise and offline: all test prefixes are available at once, one score per rally, a setting on which Section 4.4 depends. The test set provides the visible prefixes but not the labels of the strokes to be predicted; it therefore defines the deployment input distribution (Table 1, Fig. 1), while all scores are estimated by match-grouped cross-validation (Sections 4.5 and 5.1).

3.2 Dataset Statistics and Field Semantics

Table 1 summarizes the dataset. Each stroke record carries match context, hitter and receiver identifiers, and technical attributes (stroke type, placement, forehand/backhand, power, spin, position). Three semantic details shape the features. Records are written from the hitter’s perspective and the two score fields swap perspective at every stroke, so cross-stroke features must unify the perspective. Placement is a receiver-relative coordinate (1/4/7 forehand side, 2/5/8 middle, 3/6/9 backhand side; 1–3 short, 4–6 half-long, 7–9 long). The position field is valid only for the first two strokes.

3.3 Data Challenges and Structural Properties

The dataset exhibits the two conditions of Section 1 and the observation that makes TPP possible; two further properties shape the training objective and the reading of S3. (1) **Very short prefixes.** Test prefixes are far shorter than training rallies (Table 1); even after expanding training rallies into all prefixes, $K \leq 2$ accounts for 40.3%, short of the 53.3% in the test set (Fig. 1; Section 4.5). (2) **Training-unseen players.** 31 test players never appear in training (Table 1). (3) **Visible strokes of unseen players.** All of them nevertheless leave visible strokes in the test set (as hitters: mean 56.4 strokes, minimum 25);

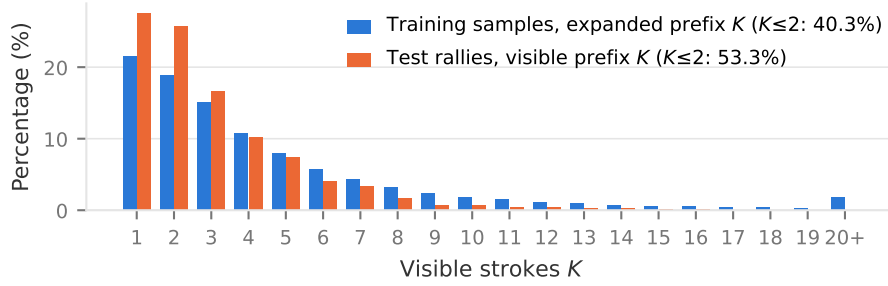


Fig. 1. Distribution of the visible prefix length K : training samples after prefix expansion versus test rallies (“20+” is a merged bin).

this is not a zero-information cold start [11] but a transductive setting whose test prefixes contain no prediction targets, which motivates TTPP. A further property, **class imbalance**, shapes the training objective (Section 4.6): among training target strokes, the largest stroke-type class outnumbers the rarest by 41:1, and placement by 79:1.

Structural property. Training rallies are almost all recorded up to the stroke on which the error occurred, and the two players hit alternately, so the player who hit the terminal stroke almost always determines the point winner (agreement with the server-wins label: 99.91%). In the training data, S3 is close to “predicting which side the rally ends on,” and its signal is tied to the timing and ownership of the terminal stroke (Section 5.2). Test prefixes never include the terminal stroke ($K \leq L - 1$), so S3 is intrinsically harder at test time than this training-set regularity suggests.

4 Method

4.1 Overall Architecture

The three tasks share one tabular formulation: a shared base feature table, task-specific additional features, a task-specific training configuration, and a separately trained GBDT model for each task. Every legal prefix position of every training rally is expanded into a sample. Its features come from the rally information up to stroke K plus cross-rally player statistics; its labels are the type and placement of stroke $K + 1$ (S1 and S2, multiclass) and the rally outcome (S3, binary). With TTPP, inference is conditioned on the deployment batch (Section 4.4), and validation is designed accordingly (Section 4.5); S3 uses rally-level weighting and S1 and S2 use class-balanced weights (Section 4.6).

4.2 Base Features

The prefix is not modeled as a sequence. Visible information is represented by fixed-length tabular features in five groups, 86 dimensions in total (Table 2).

Table 2. Base feature groups (86 dimensions)

Group	Dim.	Content
State	29	gender group, game number, stroke index, scores and margins from four perspectives, deuce and game-point flags, rally-phase bucket, visible stroke counts and attack/long-ball shares of both players
Current stroke	6	type, placement, forehand/backhand, spin, power, and position of stroke K
Previous stroke	8	attributes of stroke $K - 1$ (the next hitter’s own last stroke), its stroke-type group, and placement lane/depth
Opening strokes	23	attributes of the serve, return, and third stroke
Interactions	20	14 combinations among stroke- K attributes and with the game state, 3 previous-stroke $\times$ incoming-stroke combinations, and 3 serve-attribute combinations

Categorical fields that are undefined for short prefixes take -1 as a separate category, missing numeric values are filled with 0, and categorical vocabularies are built from each fold’s training rows and then fixed. The column dictionary is in the released code.

4.3 Player Statistics Features

Below, **recv_** denotes the next hitter (the player who receives stroke K and hits stroke $K + 1$) and **opp_** the opponent. To capture individual differences (different players hit different next strokes in the same situation), we build **43-dimensional** statistics per player (19-dim stroke-type distribution, 3 forehand/backhand, 6 spin, 4 power, 10-dim placement distribution, and 1 win rate), attached once for the next hitter (**recv_**) and once for the opponent (**opp_**), another 86 dimensions in total. This set of statistics for one player is their **player profile**; “player-statistics features” denotes the features built from it, and TTPP (Section 4.4) builds profiles at test time. Four rules apply: the source is the raw stroke table (one row per actual stroke, not multiplied by prefix expansion); distributions are relative frequencies without smoothing, and the stroke-type distribution keeps all 19 classes; players with fewer than 5 stroke rows receive no individual statistics and fall back to the global prior; and the win rate is rally-level: the share of rallies won by the server among the rallies the player served, with players who never served falling back to the global rate. The global prior is the same 43 statistics computed over all training strokes pooled together regardless of player: its stroke-type distribution, for example, is the share of each stroke type in the whole training set, and its win rate is the share of rallies won by the server in the whole training set. Training-unseen players also receive the global prior when TTPP is not used (Section 4.4). The placement model additionally uses 30 **receiver-exposure statistics** (S2 only): the distribution of where a player is hit as a receiver, and its version conditioned on

the incoming stroke-type group, estimated with empirical-Bayes shrinkage $\tilde{p}_c = (n_c + \alpha p_c^{\text{global}})/(n + \alpha)$, $\alpha = 50$. They enter in the bundled step (Table 3, row 5).

Training-row statistics are **cross-fitted** so that no sample contributes its own strokes or outcome to its own profile: each fold’s training rallies are assigned with a fixed seed to 5 inner folds, and the statistics of a training sample (including the global prior and the fewer-than-5-rows fallback) are recomputed with its inner fold removed. Inner folds are grouped by rally, so a training row’s profile may include other rallies of the same match; validation and test rows never see any outcome of the match being predicted (their sources are given in Section 4.5). This is **rally-level cross-fitting**, the main training recipe. A stricter **match-level cross-fitting** variant groups the inner folds by match, so that a training row’s profile excludes every rally of its own match; it gives training-row profiles less cross-rally support and serves as a sensitivity check in Section 5.3. The rally-level grouping is a train–inference mismatch, not a validation leak.

4.4 Test-Time Player Profiling (TTPP)

TTPP is an inference-time procedure. At test time we pool each player’s visible strokes across all rallies of the test batch and compute the same 43 statistics from them (attributes only, never any prediction target); players with no data fall back to the global prior. The two cases are:

- *Without TTPP*: a training-seen player receives the training-derived profile; a training-unseen player receives the global prior.
- *With TTPP*: a training-seen player receives the training profile extended with their visible test strokes; a training-unseen player receives a profile built from their visible test strokes.

The model is not retrained: training rows keep the cross-fitted statistics of Section 4.3, computed from training data only, and the trained GBDT is applied to the test rows with the new profiles. TTPP extends only the dimensions derived from stroke attributes: the win rate needs outcomes and keeps its training-set source, and receiver-exposure statistics are not extended. Because test prefixes are mostly $K \leq 2$, the statistics of unseen players are biased toward opening strokes; no correction is applied, and the effect by prefix length is examined in Section 5.3. A **batch-conditioned training variant**, which also adds the test strokes to the training-row statistics and retrain, performs the same under the rally-level cross-fitting of Section 4.3 but retains more of the gain under match-level cross-fitting (Section 5.3). TTPP conditions the input features on the deployment batch and therefore depends on batch (offline) inference (Section 6).

4.5 Validation Design

This section answers the validation question raised in Section 1: how to set validation up so that it reflects deployment.

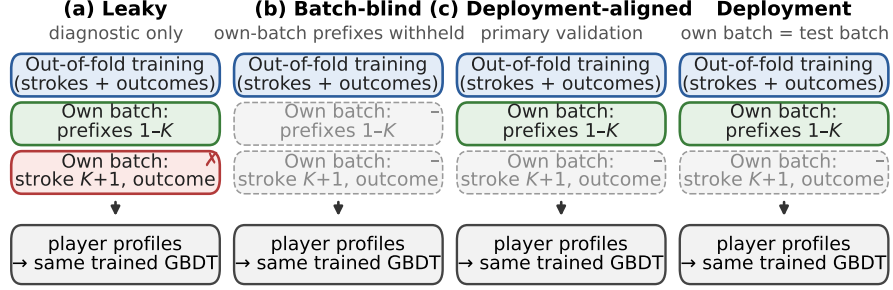


Fig. 2. Which data sources enter the validation-row player profiles under each regime, and the test-row profiles at deployment. Colored blocks are used, grey blocks are excluded; the trained model is the same in every panel. Regime (c) has the same shape as deployment; (a) additionally uses the hidden targets (illegitimate, diagnostic only); (b) withholds the batch’s own prefixes.

(1) Evaluation-unit alignment. Validation is 5-fold cross-validation grouped by match, mirroring the dataset’s train/test split (matches disjoint, 40 of 71 test players seen in training), so every validation fold contains players absent from its training folds, called **fold-unseen players** below. Each validation rally contributes **one** prefix, so that the validation unit matches the test unit (one score per rally); K is drawn once with a fixed seed from the test K distribution restricted to $K \leq L-1$ and renormalized; we call the result the sampled validation set. Because short rallies can only supply small K , the validation K distribution is skewed toward short prefixes (Section 5.1), a consequence quantified in Section 5.3.

(2) Three regimes for validation-row player profiles. Validation rallies also need player profiles, and the question is which strokes may be used to build them. The trained models are the same in every regime, because training rows always use cross-fitted training data (Section 4.3); only the validation-row profiles differ. Below, the *validation batch* means the validation fold’s rallies, which play the role of the test batch. We compare three regimes (Fig. 2). The key contrast is between (b) and (c): both exclude the validation rallies’ stroke $K+1$ and outcomes, and they differ only in whether the validation batch’s own prefixes (strokes 1– K) may be used.

(a) Leaky. Profiles are computed once over the whole training set, including the validation fold, so the profile of a validation player contains the very rally being predicted, its stroke $K+1$ and its outcome. This is the common mistake of cross-sample features (Section 2); we include it to measure how much it inflates the score.

(b) Batch-blind. Profiles use only the other four folds; the validation batch’s prefixes are not used. A fold-unseen player therefore has no profile and falls back to the global prior; TTPP has no batch prefixes to use, so its gain is zero by

construction. This is the standard out-of-fold rule, and it cannot see the method at all.

(c) Deployment-aligned. Profiles use the other four folds plus the validation batch’s prefixes (strokes 1– K of each validation rally, never stroke $K + 1$ or the outcome). A fold-unseen player therefore obtains a profile from those visible strokes, exactly as at deployment, where test-row profiles use the training data plus the test batch’s prefixes (Section 4.4). This is our main regime.

Without TTPP no batch prefixes are used, so (b) and (c) coincide; with TTPP, only (c) has batch prefixes to add.

4.6 Learner and Training Settings

Model and sample weights. Each task has its own LightGBM model [6], trained once per fold, with no further ensembling. Training samples do not all count equally; two adjustments are made. First, class balance (S1 and S2). Stroke-type and placement classes are highly imbalanced (Section 3.3), whereas Macro-F1 treats every class alike, so rare-class samples receive larger weights ($w_c = N/(C \cdot N_c)$). Second, rally-level weighting (S3 only). Each training rally is expanded into $L - 1$ samples (Section 4.1) that all carry the same label, whereas evaluation scores each rally once; each S3 sample is therefore weighted by $1/(L - 1)$, so that every rally has a total weight of 1 (effect in Section 5.2).

Selection and inference. Early stopping monitors the log-loss on the fold’s sampled validation set, and the hyperparameters were searched on the same split, so the scores in Section 5 are development-set estimates that include model selection (Section 6). For test rows, the probabilities of the five fold models are averaged. S1 and S2 predict the highest-probability class without post-processing; S3 is scored by AUC on the averaged probability.

5 Experiments and Results

5.1 Experimental Setup

Protocol. All numbers below come from the validation of Section 4.5 (the primary split, seed 158; seeds 42 and 2024 for replication, neither used for any feature, hyperparameter, or method selection, so they serve as held-out confirmation of the two core comparisons). $K \leq 2$ accounts for 74.5% of validation samples versus 53.3% of the test set (K -stratified analysis in Section 5.3); rallies with at least one fold-unseen player make up 46.6% of validation rallies (54.4% in the test set). **Attribution.** In Table 3, rows 2→3 differ only in player statistics, rows 3→4 only in the S3 weighting, and rows 5→6 only in the validation-row statistics of TTPP (regime (c); same trained models), each under identical hyperparameters; row 4→5 is a bundled step with several changes and is not attributed to any single component. Rows 1–4 and Table 6 use the initial hyperparameters; rows 5–6 and Table 5 use the tuned setting found by a search on the same split (Section 6). **Statistical notes.** Macro-F1 always uses the fixed

Table 3. Stepwise configuration comparison (evaluation-unit-aligned validation; feature groups as in Table 2; bold = best per column; $\star$ marks the player-statistics step)

Row	Configuration	S1	S2	S3	Overall
1	State (5 of 29) + current stroke (Markov baseline)	0.2629	0.1636	0.5947	0.2895
2	+ 14 stroke- K interactions	0.2568	0.1610	0.5890	0.2849
3	+ player-statistics features (Sec. 4.3) $\star$	0.3323	0.1787	0.5886	0.3221
4	+ S3 rally-level weighting (Sec. 4.6)	0.3323	0.1787	0.6028	0.3250
5	+ remaining state, previous/opening strokes, 6 further interactions, receiver exposure, tuned hyperparameters	0.3690	0.1873	0.6126	0.3450
6	+ test-time player profiling (TTPP, Sec. 4.4), regime (c)	0.4020	0.1941	0.6187	0.3622
5-	Row 5 without player-statistics features (leave-one-out)	0.3078	0.1825	0.6106	0.3182

class set; only the subgroup analysis of Table 4 scores over the classes present in each subgroup (present-class scoring). Tabulated scores are computed once over the pooled five-fold samples, and “per-fold” values are fold averages; the two can differ by 0.001–0.004 because Macro-F1 is nonlinear. Folds are not independent trials, so fold-wise sign counts (5/5 = positive in 5 of 5 folds) and bootstrap intervals (match-level, 1,000 resamples) serve only as evidence of directional robustness.

5.2 Stepwise Configuration Comparison: Player Statistics Give the Largest Single-Component Gain

Table 3 reports validation scores as features are added step by step, labeled by the feature groups of Table 2. Predicting the majority class for every sample gives a Macro-F1 of 0.026 (S1) and 0.045 (S2), far below the Markov baseline in row 1.

We attribute the final configuration’s score in two ways. The primary evidence is a leave-one-out check under the tuned hyperparameters, which is order-independent: removing all player-statistics features from the final model (row 5-) lowers Overall from 0.3450 to 0.3182 (−0.0268, 5/5 folds), the largest drop of any single component and larger than the TTPP step measured in the same setting (+0.0171). The stepwise chain of Table 3 shows the same ordering as features are added: relative to the preceding row, player statistics ($\star$) add +0.037 Overall (+13.1%) and +0.075 on S1 (+29.4%), positive in 5/5 folds, with a bootstrap 95% CI of [+0.028, +0.045]. On the other two splits the step is +0.0382 and +0.0419, so it is positive in all 15 folds across the three splits. Rows 1–4 use the initial hyperparameters and row 5 adds several components at once (+0.0200, not attributed to any single one), so the chain is reported for context rather than as the basis of attribution. The 14 stroke- K interactions are neutral to slightly

Table 4. Stratified structure of the TTPP gain (final configuration; present-class scoring; 0 = both players seen, 2 = both unseen)

Fold-unseen players in rally	Rallies	Δ Type (c)	Δ Place. (c)
0	8,011 (53.4%)	−0.0048	−0.0005
1	6,153 (41.0%)	+0.0762	+0.0092
2	831 (5.5%)	+0.1529	+0.0147

negative (−0.0046) and are retained in the final model. S3 is almost unaffected by player statistics (−0.0004), consistent with the terminal-stroke structure of Section 3.3; rally-level weighting is the most effective isolated modification for S3 (+0.0142, 5/5).

5.3 TTPP and Training-Unseen Players

Main effect. Under the deployment-aligned regime (c), TTPP raises Overall from 0.3450 to **0.3622 (+0.0171)**, positive in 5/5 folds (S1 and S3 also 5/5), with a bootstrap 95% CI of [+0.010, +0.025], and +0.0158 and +0.0132 on the other two splits, positive in all 15 folds across the three splits. Under the batch-blind regime (b) the gain is zero by construction (Section 5.4). Under rally-level cross-fitting, retraining adds nothing: the batch-conditioned training variant of Section 4.4 reaches 0.3610 on the primary split, and its difference from TTPP is −0.0012, +0.0014, and +0.0009 on the three splits, within noise and without a consistent sign.

Where the gain comes from. Table 4 stratifies rallies by the number of fold-unseen players (Macro-F1 over the classes present in each subgroup, i.e., present-class scoring; class and K distributions differ across subgroups, so absolute values are not directly comparable). Stroke-type and placement gains both increase monotonically with the number of unseen players. Rallies with both players seen are slightly negative (stroke type −0.0048); we attribute this to the fact that under (c) TTPP also extends the statistics source of seen players with the validation batch’s prefixes (the test batch at deployment). The gain thus tracks exposure to unseen players, as intended; this is a heterogeneity analysis, not an isolation of the mechanism, and the both-unseen subgroup (831 rallies) is indicative only.

Prefix length. Stratified by prefix length, the gain under (c) is positive in all four K buckets ($K = 1$ +0.0163, $K = 2$ +0.0175, $K = 3$ –4 +0.0258, $K \geq 5$ +0.0074), and reweighting to the test K distribution moves it from +0.0171 to +0.0169, so shorter validation prefixes do not inflate it.

Sensitivity to profile construction. Under match-level cross-fitting (Section 4.3), the TTPP gain shrinks to +0.0057, +0.0024, and +0.0034 on the three splits (11/15 folds; every bootstrap interval crosses zero), whereas the batch-conditioned training variant retains gains of +0.0103, +0.0086, and +0.0105

Table 5. Effect of the information policy for player statistics on validation conclusions (final pipeline) ‡ Identical by construction: TTPP has no batch prefixes to add under (a) or (b), and without TTPP none are used under (c).

Variant	Leaky (a)	Batch-blind (b)	Deployment- aligned (c)
Final configuration without TTPP (Table 3 row 5)	0.3719	0.3450	0.3450‡
Final configuration + TTPP (Table 3 row 6)	0.3719‡	0.3450‡	0.3622

(13/15; intervals exclude zero): when training-row statistics have less support, retraining with the batch’s prefixes does help.

5.4 Validation-Regime Experiment: How the Information Policy for Player Statistics Changes Validation Conclusions

Table 5 compares the three regimes for validation-row profiles (Section 4.5, Fig. 2) on the final feature configuration (Table 3 rows 5–6, same split); all three evaluate the same trained models, and the conclusions come in two layers.

First layer, (a) versus (b): the final configuration scores 0.3719 under (a) and 0.3450 under (b), higher under (a) in all 5 folds (per-fold +0.0267). Letting validation-row profiles see the validation rallies’ own later strokes and outcomes systematically inflates the score estimate. TTPP changes nothing under either regime: under (a) the profiles already contain every stroke of the validation batch, and under (b) they may contain none. Second layer, (b) versus (c): each fold’s validation rallies involve 18–28 fold-unseen players, none of whom has a profile under (b), whereas under (c) all of them obtain one from the validation batch’s prefixes; (b) withholds information that deployment does have, and once the policy supplies it, the TTPP gain goes from zero to +0.0171. This is a difference in information policy, not merely in how the evaluation is carried out; under (a) or (b) there is no empirical basis for retaining TTPP.

5.5 Comparison with a Deep Sequence Model

Under the same protocol, the GBDT with player statistics (Table 3 row 3) outperforms ShuttleNet [16], the reference architecture for stroke-level prediction in racket sports (Table 6). Neither side underwent a systematic hyperparameter search, and ShuttleNet represents players by learned embeddings, so Table 6 compares each architecture with its own player representation.

The advantage is positive in all 5 folds and appears with the addition of player statistics (before it, ShuttleNet has the higher S1). We **conjecture** that the gap arises because player statistics can be supplied directly as input features, whereas learned embeddings receive no training signal for the 46.6% of validation rallies that involve fold-unseen players. We ran no controlled experiment separating architecture from features, so the conclusion is limited to this implementation.

Table 6. Comparison with a sequence model (S1 on 15 classes; the GBDT rows are the Table 3 row-1 and row-3 settings without S3 rally weighting; bold = best per column) † ShuttleNet trained by us on the same split and sampling setting; S3 from its own rally-outcome head; S1 rescored exactly from 19-class to 15-class scoring.

Model	S1	S2	S3	Overall
ShuttleNet (sequence model)†	0.2816	0.1446	0.5842	0.2873
LightGBM Markov baseline	0.2629	0.1636	0.5947	0.2895
LightGBM + player statistics	0.3323	0.1787	0.5886	0.3221

6 Discussion

What we learned. With only one or two visible strokes, the rally itself says little about what comes next. What helped most was knowing who is hitting: a player’s cross-rally statistics gave the largest single gain (Table 3 and the leave-one-out check), and only after adding them did the GBDT overtake ShuttleNet on stroke type, each model with its own player representation (Section 5.5).

When TTPP helps. TTPP relies on batch (offline) inference with all prefixes visible at once, on a substantial share of test players unseen in training but observed in the batch, and on player distributions being stable across rallies, an assumption we do not test. Its gain concentrates in rallies with unseen players (Table 4) and, under the main training recipe, needs no retraining; when training-time profiles have less cross-rally support (match-level cross-fitting), the inference-only gain shrinks to $+0.002$ – 0.006 with intervals crossing zero, while the batch-conditioned training variant keeps about $+0.01$ (Section 5.3). Its on-line, one-rally-at-a-time form is unverified.

The validation lesson. Whenever a feature is computed across samples, validation must specify its information set explicitly and align it with deployment, following two principles: do not supply information that does not exist at deployment ($((a) \rightarrow (b))$), and do not remove legitimate information that deployment does have ($((b) \rightarrow (c))$). The second decides whether local validation can see a method’s effect at all. The lesson is a case study in this setting; systems that rely on target encoding [7], user or item aggregate statistics, or similar cross-sample features may face analogous risks, a testable hypothesis rather than a generalized conclusion.

Limitations. Two matter most. First, hyperparameter search and early stopping used the validation folds of the same split, so absolute scores may be optimistic; the two core comparisons were replicated on two further splits with the same hyperparameters (positive in all 10 folds), which mitigates split-specific chance but not the selection bias of repeated validation feedback. Second, the comparison with the sequence model does not separate architecture from player representation, and the regime comparison of Table 5 is on one dataset and one split. A minor point: the stratified analysis uses present-class scoring with a small both-unseen subgroup.

7 Conclusion

For table tennis next-stroke prediction under very short prefixes and a high share of training-unseen players, we proposed a GBDT pipeline augmented with player statistics for both the next hitter and the opponent and with Test-Time Player Profiling (TTPP), evaluated with a validation protocol aligned to the information available at deployment. Cross-rally player statistics give the largest single-component gain (+0.037 in the stepwise chain and -0.027 when removed from the final model, positive in all 15 folds of three splits), and TTPP adds +0.017 under the deployment-aligned regime, concentrated in rallies with unseen players and, under the main recipe, without retraining the model. In settings like ours, validation of cross-sample statistical features should be designed together with their information policy: letting validation-row statistics use the validation rallies’ own later strokes and outcomes inflates scores, and withholding the validation batch’s prefixes that deployment does have leaves no TTPP gain to observe. Future work: a factorial architecture $\times$ player-statistics design, prefix-length-conditioned statistics for unseen players, and replication of the information-policy lesson on other datasets.

Acknowledgments

We thank Mei-Chen Yeh of National Taiwan Normal University for advice on the writing of this paper.

This study uses the dataset of the Ministry of Education AI CUP 2026 “Table Tennis Tactic and Outcome Prediction from Sequential Data” competition and outcomes of the project “From Taiwan to the World: Integrated Research on Intelligent Table Tennis Combining Information Technology and Sports Medicine.” We thank the Department of Information and Technology Education, Ministry of Education; the AI Competition and Labeled Data Acquisition Project Office; the Department of Computer Science and Information Engineering, National Central University, for the problem and data; and the AIda platform. The AI CUP 2026 dataset was released for academic research.

Code, the validation-protocol implementation, the feature dictionary, and the configuration files for every table are at <https://github.com/Eason20050201/ttpp-table-tennis>.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. AIda: AI CUP 2026: Table tennis tactic and outcome prediction from sequential data. <https://www.aidea-web.tw/topic/3f9662e8-9d18-4c6a-9332-103eded3a399> (2026)
2. Grinsztajn, L., Oyallon, E., Varoquaux, G.: Why do tree-based models still outperform deep learning on typical tabular data? In: Advances in Neural Information Processing Systems, Datasets and Benchmarks Track (2022)

3. Hsu, M.H., Kondrič, M., Fan-Chiang, I.Y., Wu, J.L.: A new table tennis match stroke forecasting method using transformer-based deep neural networks. *Montenegrin Journal of Sports Science and Medicine* **14**(1), 45–55 (2025). <https://doi.org/10.26773/mjssm.260306>
4. Ibh, M., Graßhof, S., Witzner Hansen, D.: A stroke of genius: Predicting the next move in badminton. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. pp. 3376–3385 (2024)
5. Kaufman, S., Rosset, S., Perlich, C., Stitelman, O.: Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data* **6**(4), 15:1–15:21 (2012)
6. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: A highly efficient gradient boosting decision tree. In: *Advances in Neural Information Processing Systems*. vol. 30, pp. 3146–3154 (2017)
7. Micci-Barreca, D.: A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter* **3**(1), 27–32 (2001)
8. Na, D., Xue, Q.: AI-driven tactical recommendations for table tennis: Decision optimization with probabilistic interaction model and technical quantification system. *Expert Systems with Applications* **293**, 128600 (2025)
9. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A.V., Gulin, A.: CatBoost: Unbiased boosting with categorical features. In: *Advances in Neural Information Processing Systems*. vol. 31, pp. 6639–6649 (2018)
10. Rothe, F., Liu, R., Lames, M.: Estimating the relevance of first offensive shot tactics in table tennis via simulation based on a finite Markov chain model. *International Journal of Computer Science in Sport* **24**(1), 1–16 (2025)
11. Schein, A.I., Popescul, A., Ungar, L.H., Pennock, D.M.: Methods and metrics for cold-start recommendations. In: *Proceedings of the 25th Annual International ACM SIGIR Conference*. pp. 253–260 (2002)
12. Seong, M., Oh, J., Kim, S.: MuLMINet: Multi-layer multi-input transformer network with weighted loss. *arXiv:2307.08262* (2023)
13. Vapnik, V.N.: *Statistical Learning Theory*. Wiley, New York (1998)
14. Wang, D., Shelhamer, E., Liu, S., Olshausen, B., Darrell, T.: Tent: Fully test-time adaptation by entropy minimization. In: *International Conference on Learning Representations (ICLR)* (2021)
15. Wang, W.Y., Du, W.W., Peng, W.C., Ik, T.U.: Benchmarking stroke forecasting with stroke-level badminton dataset. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)* (2024)
16. Wang, W.Y., Shuai, H.H., Chang, K.S., Peng, W.C.: ShuttleNet: Position-aware fusion of rally progress and player styles for stroke forecasting in badminton. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. vol. 36, pp. 4219–4227 (2022)
17. Zhou, Z., Zhang, H.: A visible analysis approach for table tennis tactical benefit. *Journal of Sports Science and Medicine* **21**, 517–527 (2022)